

Math Formulation: GAN Generator with Frozen Permeability Classifier and GRPO Training — Continuation

S Hariharan, J Shrenik Reddy, Ansa N

This continues directly from Section 7 of the working draft (Algorithm Sketch), formalizing the three items listed under “Open Items to Formalize Next.” Equation numbers continue from (8); section numbers continue from 7.

1 Parameterizing $\pi_\theta(a_t \mid a_{<t}, z)$

Item 1 asked what an “action” a_t actually is. This depends on whether G_θ is autoregressive (SELFIES) or one-shot (MolGAN-style atom/bond tensors). Both are formalized below so the choice can be made explicit in the implementation and the rest of the math (Eqs. 1–8) specialized accordingly.

1.1 Case A: Autoregressive SELFIES decoding

Let $\mathcal{V}$ be the SELFIES token vocabulary (including a terminal [EOS] token) and let f_θ be a sequence model (RNN/Transformer) conditioned on the latent z . At step t ,

$$\pi_\theta(a_t \mid a_{<t}, z) = \text{softmax}(f_\theta(a_{<t}, z))_{a_t}, \quad a_t \in \mathcal{V}, \quad (1)$$

and decoding terminates at the first $t = T$ with $a_T = \text{[EOS]}$ (or a fixed maximum length $T_{\max}$). T is therefore a random variable determined by the rollout itself, not a hyperparameter. The molecule is obtained by the deterministic SELFIES→graph parser, $m = \text{Parse}(a_1, \dots, a_T)$, so $\pi_\theta(m)$ in Eq. 1 is exactly the product over the *realized* T :

$$\pi_\theta(m) = \prod_{t=1}^T \pi_\theta(a_t \mid a_{<t}, z), \quad \log \pi_\theta(m) = \sum_{t=1}^T \log \pi_\theta(a_t \mid a_{<t}, z). \quad (2)$$

Because Parse is guaranteed syntactically valid by construction of the SELFIES grammar (every token string maps to *some* molecule), Valid(m) in Section 4 is checking chemical/RDKit sanitization validity, not SELFIES syntax validity — these are different failure modes and should not be conflated in the implementation.

The per-token ratio $r_t(\theta)$ from Eq. 2 is unchanged and applies token-by-token; the clipped objective (Eq. 7) is naturally a sum over $t = 1, \dots, T$ per sampled molecule.

1.2 Case B: One-shot atom/bond tensor (MolGAN-style)

Here G_θ maps z directly to two tensors in a single forward pass: a node/atom-type tensor $X \in \mathbb{R}^{N_{\max} \times |\mathcal{A}|}$ (logits over atom types per node slot, including a “no atom” padding type) and an edge/bond-type tensor $E \in \mathbb{R}^{N_{\max} \times N_{\max} \times |\mathcal{B}|}$ (logits over bond types per node pair, including “no

bond”), where $N_{\max}$ is a fixed maximum atom count. There is no notion of a sequential decision $a_1, \dots, a_T$; instead there is a single joint action

$$a = (x_1, \dots, x_{N_{\max}}, e_{12}, e_{13}, \dots, e_{(N_{\max}-1)N_{\max}}), \quad (3)$$

with each $x_i \in \mathcal{A}$ sampled from $\text{Categorical}(\text{softmax}(X_i))$ and each $e_{ij} \in \mathcal{B}$ sampled from $\text{Categorical}(\text{softmax}(E_{ij}))$, independently conditioned on z (the joint distribution over the tensor entries factorizes given z , even though the resulting graph is not independent of itself once parsed and sanitized). The action probability is then

$$\pi_{\theta}(a \mid z) = \prod_{i=1}^{N_{\max}} \pi_{\theta}(x_i \mid z) \cdot \prod_{1 \leq i < j \leq N_{\max}} \pi_{\theta}(e_{ij} \mid z). \quad (4)$$

In the notation of Eq. 1, this is the degenerate $T = 1$ case: the “sequence” collapses to one joint categorical draw, and $\pi_{\theta}(m) = \pi_{\theta}(a \mid z)$ directly (no autoregressive conditioning on previous atoms/bonds). Consequently the ratio $r_t(\theta)$ in Eq. 2 is not indexed by t at all; it becomes a single per-molecule ratio

$$r(\theta) = \frac{\pi_{\theta}(a \mid z)}{\pi_{\theta_{\text{old}}}(a \mid z)}, \quad (5)$$

and the clipped surrogate (Eq. 7) drops its inner $\mathbb{E}_t[\cdot]$, reducing to $L_i^{\text{CLIP}}(\theta) = \min(r(\theta)A_i, \text{clip}(r(\theta), 1 \pm \epsilon)A_i)$ evaluated once per sampled molecule i , not once per token.

Note on differentiability. MolGAN-style generators are often trained with a Gumbel-softmax relaxation so that samples remain differentiable and the discriminator loss can backpropagate through X, E directly. That reparameterization path is *not* needed here: because discrete molecule construction already routes through the non-differentiable RDKit sanitization step (Section 2), the whole pipeline is trained with the REINFORCE/PPO score-function estimator via $R(m)$, not via a straight-through gradient. If a Gumbel-softmax relaxation is used at all, it should be restricted to stabilizing D_{ϕ} ’s own adversarial loss in Eq. 3–4 (which is a separate, directly differentiable objective), and kept out of the θ update path to avoid mixing two different gradient estimators for the same parameters.

1.3 Which case governs $r_t(\theta)$ downstream

Eqs. 7–8 should be read as written (indexed by t) under Case A, and with t collapsed to a single joint action under Case B via Eq. (5). This is the concrete fork that Section 7’s Algorithm Sketch (step 2, “Generate molecules”) needs to commit to before implementation; nothing else in Sections 3–6 depends on the choice.

2 Fixed vs. Annealed Reward Weights

Item 2 asked whether w_D, w_C, w_Q, w_S in Eq. 5 are fixed or annealed. Write k for the generator training step (the outer loop counter in Section 7, step 9).

2.1 Fixed-weight baseline

The simplest choice keeps w_D, w_C, w_Q, w_S constant for all k , subject only to $w_D, w_C, w_Q, w_S \geq 0$. This is the version implicitly used in Eq. 5 and is the right default to implement first, since it has no extra hyperparameters beyond the weights themselves.

2.2 Annealed schedule

Motivated by the concern in the draft (establish validity/realism first, then shift weight to C), define each weight as a function of k interpolating between an initial and final value over an annealing horizon k_{anneal} :

$$w_{\bullet}(k) = w_{\bullet}^{(0)} + (w_{\bullet}^{(f)} - w_{\bullet}^{(0)}) \cdot \lambda(k), \quad \lambda(k) = \min\left(\frac{k}{k_{\text{anneal}}}, 1\right), \quad (6)$$

for $\bullet \in \{D, C, Q, S\}$, with e.g. $w_D^{(0)} \gg w_C^{(0)}$ and $w_D^{(f)} \ll w_C^{(f)}$ to realize the “validity first, then permeability” curriculum. $\lambda(k)$ can equally be a cosine schedule, $\lambda(k) = \frac{1}{2}(1 - \cos(\pi \min(k/k_{\text{anneal}}, 1)))$, if smoother transitions are preferred; the choice does not change any other equation, since $w_{\bullet}(k)$ only enters Eq. 5 as a per-step constant.

2.3 Validity-gated curriculum switch

An alternative to a fixed k_{anneal} is to gate the transition on measured performance rather than wall-clock steps. Let $\rho_{\text{valid}}(k) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\text{Valid}(m_i)]$ be the fraction of valid molecules in step k ’s sampled group. Replace $\lambda(k)$ in Eq. (6) with

$$\lambda(k) = \mathbb{I}[\bar{\rho}_{\text{valid}}(k) \geq \tau], \quad \bar{\rho}_{\text{valid}}(k) = \frac{1}{W} \sum_{j=k-W+1}^k \rho_{\text{valid}}(j), \quad (7)$$

a hard switch once the W -step moving average of validity rate exceeds threshold τ (e.g. $\tau = 0.9$). This ties the curriculum to the generator’s actual competence at producing sanitizable molecules rather than to an arbitrarily chosen step count, at the cost of one more hyperparameter pair (τ, W) . Either schedule is a drop-in replacement for the constant weights in Eq. 5; no other equation in Sections 4–8 needs to change.

3 Discriminator/Generator Update Ratio n_D and Reward Non-Stationarity

Item 3 asked for a formal treatment of n_D (Section 7, step 8) and its effect on the reward’s non-stationarity, since D_{ϕ} — unlike the frozen classifier C — is itself being trained.

3.1 Why this is a genuine non-stationarity, and why C is exempt

Write $R_k(m)$ for the reward function actually used at generator step k , i.e. Eq. 5 with D_{ϕ} replaced by D_{ϕ_k} , the discriminator parameters after its k -th block of n_D updates. Because ϕ_k changes with k , R_k is a genuinely time-varying reward: the same molecule m can receive a different $w_D D_{\phi_k}(m)$ term at step k than it would have at step $k' \neq k$, even though nothing about m changed. This is exactly the standard GAN dynamics problem transplanted into the reward function. By contrast C is frozen ($\nabla_{\phi} C = 0$, Section 4), so the $w_C C(m)$ term is stationary in k by construction — it is only the D_{ϕ} term that introduces non-stationarity.

3.2 Two-timescale formalization

The standard way to make convergence arguments tractable here (following the two-timescale update rule (TTUR) analysis used for GANs) is to treat ϕ as evolving on a *faster* timescale than

θ : choose $n_D \geq 1$ discriminator gradient steps per generator step (or a discriminator learning rate $\eta_\phi \gg \eta_\theta$) such that, on the timescale over which θ changes appreciably, ϕ_k can be treated as approximately equilibrated to the current generator distribution p_{θ_k} :

$$\phi_k \approx \phi^*(\theta_k) := \arg \min_{\phi} L_D(\phi; \theta_k), \quad (8)$$

with L_D as in Eq. 3 (or Eq. 4). Under this quasi-static assumption, $R_k(m) \approx R(m; \theta_k)$ becomes a (slowly) θ -dependent reward rather than an arbitrarily-moving target, which is the condition under which the GRPO update in Section 5–6 can be interpreted as ordinary policy-gradient ascent on a well-defined, slowly-drifting objective at each step, rather than pursuing a reward that has no stationary point. This is an assumption, not a guarantee: it degrades as $n_D \rightarrow 1$ (discriminator barely updated before the next generator step) and improves as n_D grows, at the usual cost of more discriminator compute per generator step.

3.3 Effect on the group-relative advantage

The advantage A_i (Eq. 6) is computed *within* a single step k , using μ_{R_k}, σ_{R_k} over the current group only — so intra-group ranking of candidates is unaffected by D_ϕ drift across steps: every candidate in the group is scored by the same D_{ϕ_k} snapshot. The non-stationarity instead shows up *across* steps, in two ways worth tracking empirically rather than in the group statistics themselves:

1. **Signal magnitude drift.** As D_ϕ sharpens over training (its output distribution typically becomes less entropic, pushing $D_\phi(m)$ toward $\{0, 1\}$), the *variance* of the $w_D D_\phi(m)$ term across a group can grow or shrink relative to the fixed-variance $w_C C(m)$ term, changing the effective relative weight the two terms have on A_i even if w_D, w_C are held literally constant (Section 9.1). This is worth logging ($\text{Var}_i[D_{\phi_k}(m_i)]$ per step) as a diagnostic.
2. **Off-policy staleness within an update.** Section 6 reuses the same sampled group for several PPO epochs/minibatches before resampling (standard PPO reuse pattern). If ϕ is also updated during that same window (i.e. n_D discriminator steps interleaved with the K generator epochs of step 7 in Section 7), the reward used to compute A_i at the start of the window becomes stale relative to ϕ by the end of it. The safest choice is to freeze ϕ for the duration of a generator update window (i.e. perform all n_D discriminator steps either strictly before or strictly after the K -epoch PPO block, never interleaved with it), so that A_i is computed once against a single fixed ϕ snapshot per group, consistent with the frozen- C treatment in Section 4.

3.4 Practical implication for choosing n_D

Combining 10.2 and 10.3: n_D should be chosen large enough that Eq. (8) is a reasonable approximation (discriminator meaningfully ahead of the generator, as in standard WGAN-GP practice, e.g. $n_D \in [1, 5]$), but the discriminator update block should be scheduled *outside* the PPO reuse window from Section 6 rather than concurrently with it, so that R_k — and hence A_i — is well-defined as a single fixed function for the entire duration over which it is used to compute the clipped objective in Eq. 7.